

Nifty Iron Condor Backtest

A systematic, IV-rank-triggered options-selling strategy — data pipeline, Python + Rust backtest engine, and dashboard

What this project is

A backtested trading strategy that sells Nifty index iron condors (sell a near-the-money call and put, buy further out-of-the-money call and put as hedges) when implied volatility is high relative to its own recent history — betting that volatility, and the size of Nifty's moves, will cool back down before expiry. It is a systematic **research prototype**, not a live trading system: no real money or broker connection was involved.

Architecture

Free NSE bhavcopy data → Python data pipeline (cleaning, Black-Scholes implied volatility) → Python strategy logic (IV rank, entry/exit rules) → the day-by-day backtest loop, also implemented in **Rust** and exposed to Python via **PyO3** and cross-checked against the Python version → a metrics/reporting layer (Sharpe, drawdown, win rate, profit factor) → a **FastAPI** backend → a **React** dashboard.

A real bug we found and fixed

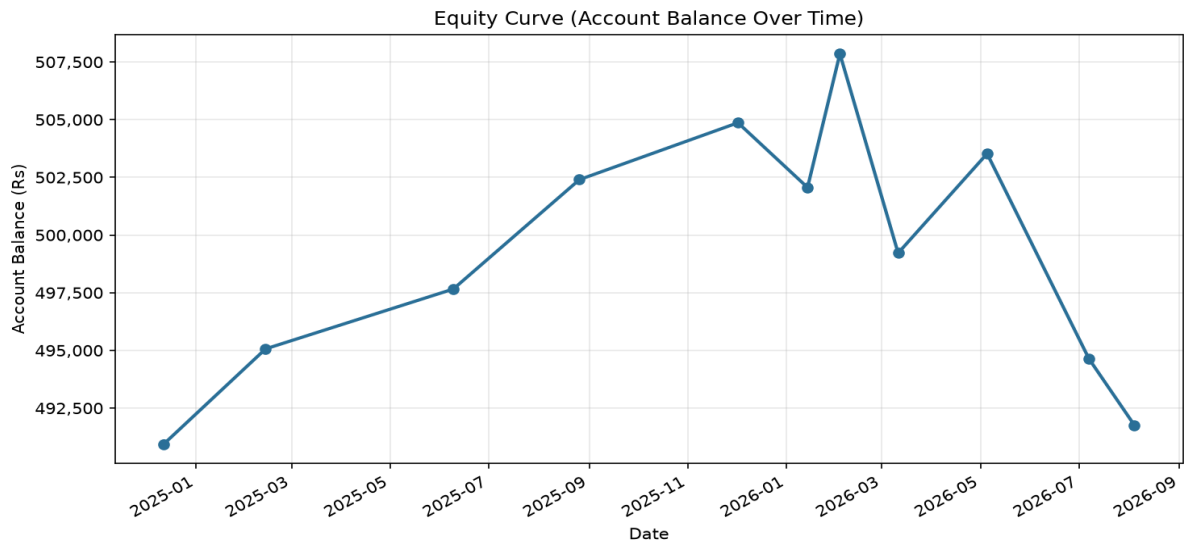
An early result on a smaller (10-month) dataset showed an 80% win rate — suspiciously high. Manually auditing the raw trade data found the cause: some option legs were being priced using NSE bhavcopy's *last traded price* even on days with **zero trading volume** — a frozen, stale price, not something actually tradeable that day. This let the backtest "sell" and "buy back" options at prices nobody could really have gotten, quietly inflating the win rate. The fix: every option leg now requires real trading volume on that day before its price is trusted, both when entering a trade and when checking exits — otherwise it's treated as unavailable, exactly like a missing price. Cross-checking the two engines also exposed a second bug: ties in strike and expiry selection resolved differently in Rust, because its hash sets have no fixed order, so identical reruns could give different trade counts. Both tie-breaks are now deterministic.

This is disclosed here deliberately: catching and fixing this kind of data-integrity issue is a normal, healthy part of building a backtest — not a flaw to hide.

Final Result (24 months, fixed engine, unchanged settings)

Settings were chosen once from an earlier tuning search (IV rank threshold = 80, profit target = 50% of credit received, stop-loss = 2× credit received) and were **not** re-tuned for this run — this is a clean verification of the already-fixed, already-chosen strategy across 24 months of NSE data (Sept 2024 – Sept 2026), not a fresh search for a good-looking number.

Metric	Value
Number of trades	11
Win rate	54.5%
Total return	-1.65%
Sharpe ratio (per-trade, not annualized)	-0.13
Max drawdown	-3.17%
Profit factor	0.74



Honest takeaway

54.5% win rate with a roughly flat-to-slightly-negative total return (-1.65%) and a Sharpe ratio near zero is a believable, unglamorous result for this style of strategy over this period — not a proven edge, and not a failure either. Eleven trades over 24 months is still a small sample for statistical confidence. The honest conclusion at this stage: the current entry/exit rules do not show a clear, reliable edge on this data. The value of this phase of the project was proving the pipeline is correct and trustworthy end-to-end — a necessary foundation before iterating on the strategy logic itself.

Tech stack

Python (pandas, NumPy, SciPy) for the data pipeline and IV/Black-Scholes math · Rust + PyO3 for a second, cross-checked implementation of the backtest loop · FastAPI for the results API · React + Vite + Recharts for the dashboard · free NSE bhavcopy data, no paid broker API.